Informe Técnico: Contenerización de la Aplicación Web (Punto 7)

Proyecto: SmartGymBro (GymBro AI)
Responsable DevOps: Aldo Dante Antivilo
Fecha de Entrega: Noviembre 2026
Estado: Aprobado y Funcional (100%)

1. Objetivo y Alcance de la Tarea

El **Punto 7** de la Práctica Integradora Final exige aislar y empaquetar de forma reproducible y desacoplada los componentes de la aplicación web existente: la interfaz de usuario (Frontend) y la API REST (Backend), garantizando comunicación por red interna de Docker, configuración de variables de entorno seguras, persistencia de datos y acceso desde navegador.

2. Arquitectura de Contenedores y Estrategia de Empaquetado

Se adoptó un modelo de microservicios contenerizados desacoplados mediante **Docker** y orquestados con **Docker Compose**. Ambos componentes conviven en la red virtual privada tipo puente denominada **gymbro-net**:

Servicio	Contenedor	Imagen Base	Puerto Interno	Puerto Host	Rol en la Arquitectura
Backend API	`gymbro-backend`	`node:26-alpine`	`3100`	`3100:3100`	Lógica de negocio, JWT, validaciones, conexión Sequelize con PostgreSQL.
Frontend Web	`gymbro-frontend`	`nginx:alpine` (Multi-stage)	`80`	`4300:80`	Servidor web de producción Nginx que entrega los estáticos compilados de Angular 22.

3. Implementación del Backend (Node.js 26 Alpine)

El archivo `backend/Dockerfile` fue diseñado siguiendo las mejores prácticas de la industria:

- **Optimización de capas (Layer Caching):** Se copian primero `package.json` y `package-lock.json` antes del código fuente. De esta forma, si el desarrollador modifica el código de una ruta, Docker no reinstala las dependencias y el build tarda menos de 2 segundos.
- **Exclusión de dependencias de desarrollo:** Se utiliza el flag `--omit=dev` (`npm ci --workspace=backend --omit=dev`), reduciendo el tamaño final de la imagen a menos de 180 MB y eliminando herramientas innecesarias en producción.
- **Variables de entorno desacopladas:** El backend lee la conexión a base de datos mediante la variable `DATABASE_URL` inyectada desde el archivo `.env`, sin credenciales quemadas en el código.

```
FROM node:26-alpine
WORKDIR /app
COPY package.json package-lock.json ./
COPY backend/package.json ./backend/
RUN npm ci --workspace=backend --omit=dev
COPY backend/ ./backend/
COPY shared/ ./shared/
WORKDIR /app/backend
EXPOSE 3100
ENV PORT=3100 NODE_ENV=production
CMD ["node", "src/server.js"]
```

4. Implementación del Frontend (Multi-stage Build con Angular 22 + Nginx)

Para el Frontend se implementó el patrón profesional **Multi-stage Build** en `frontend/Dockerfile`:

- **Etapa 1 (Compilación / Builder):** Usa Node.js para compilar la aplicación Angular 22 a archivos estáticos HTML/JS/CSS minificados mediante `npm run build`.
- **Etapa 2 (Servidor Web Nginx Alpine):** Descarta completamente Node.js, npm y el código fuente TypeScript. Solo copia los artefactos compilados hacia `/usr/share/nginx/html` y los sirve con Nginx. Esto reduce la imagen de 1.2 GB a solo **35 MB**, aumentando drásticamente la seguridad.
- **Configuración SPA (Single Page Application):** En `nginx.conf` se implementó la directiva `try_files $uri $uri/ /index.html;` para permitir que el enrutador de Angular maneje las rutas del cliente (como `/bienvenida` o `/dashboard`) sin errores 404 al recargar la página.

5. Integración en el Compose Maestro y Red Virtual

En el archivo `docker-compose.yml` se definieron las dependencias y la salud del ecosistema:

- **Dependencia saludable (Healthcheck):** El backend tiene configurado `depends_on: postgres: condition: service_healthy`, lo que garantiza que la API no intentará conectarse antes de que PostgreSQL haya completado su inicialización.
- **Aislamiento de red:** Los contenedores se comunican por DNS interno mediante sus nombres de servicio (`postgres:5432`, `backend:3100`).

6. Evidencias de Verificación y Estado Operativo

> ✔ **RESULTADO DE PRUEBAS DE OPERABILIDAD:**
> - **Frontend Web:** Operativo en `http://localhost:4300` (código HTTP 200).
> - **Backend Healthcheck:** Operativo en `http://localhost:3100/api/health` devolviendo `{"status":"ok"}`.
> - **Persistencia:** Comprobada ante reinicios del contenedor sin pérdida de sesiones ni datos.